DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 1
Reliable, Scalable, and Maintainable Applications

The three governing concerns when you compose databases, caches, indexes, and processors into a data system

Part I: Foundations · Illustrated · Interview-ready

Martin Kleppmann · Source: claude-skills / ch01-reliable-scalable-maintainable.md

Table of Contents

1. Core Idea

Data-intensive applications are composed from general-purpose building blocks — databases, caches, search indexes, stream/batch processors — stitched together by application code. Once you do that, you become a **data system designer**.

Figure 1-1 — A composite data system

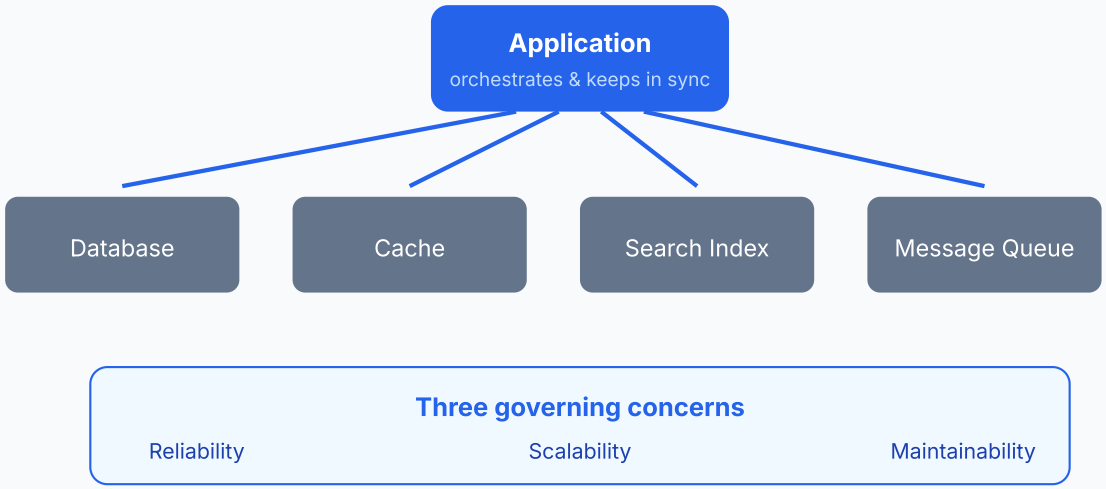

Figure 1.1 — Composing building blocks creates a data system; Part III systematizes keeping derived stores in sync.

Your job shifts from "using a database" to designing a data system. Kleppmann's frame: evaluate every design through reliability (works correctly under adversity), scalability (reasonable coping as load grows), and maintainability (many people can work on it productively over time).

2. Framework: The Three Concerns

Reliability / Scalability / Maintainability — Kleppmann's top-level checklist for any data system design discussion, before diving into technology choices.

When	Any system design discussion — use as the top-level checklist before technology choices.
How	**(1) Reliability** — system works correctly (right function, desired performance) even under adversity: hardware faults, software faults, human error. **(2) Scalability** — as load grows (data volume, traffic, complexity), there are reasonable ways to cope; never a yes/no label. **(3) Maintainability** — many people over time (engineering + ops) can work on it productively via operability, simplicity, evolvability.
Why it works	Separates orthogonal risks so you don't over-invest in one (e.g., scaling machinery) while ignoring another (e.g., ops toil).
Failure mode	Treating them as buzzwords instead of asking the concrete questions behind each.

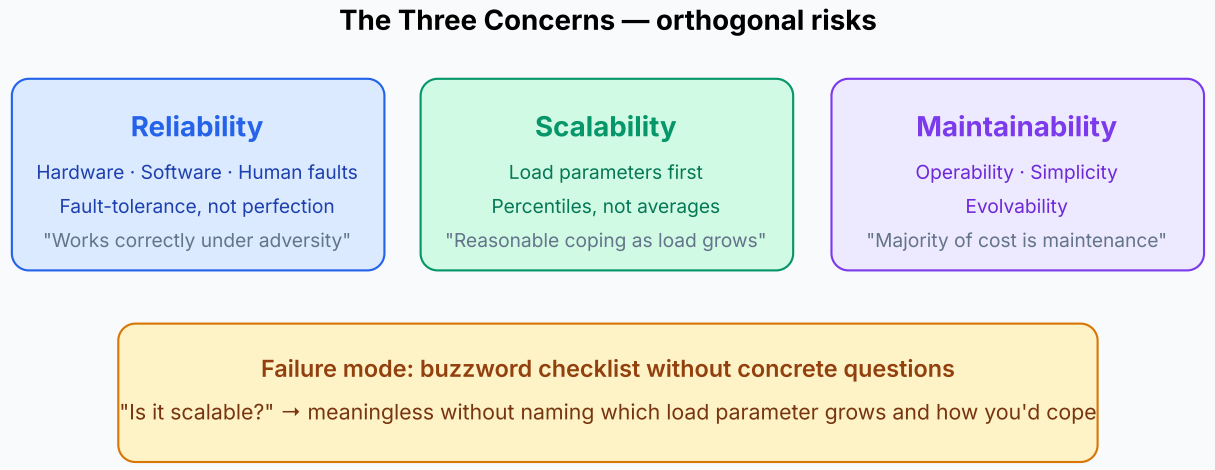

Figure 2.1 — Three orthogonal concerns; each demands different design questions.

Reliability: fault vs failure

Fault

One component deviating from spec — disk crash, bug, typo in config.

Failure

Whole system stops providing the required service to the user.

Design goal

Build fault-tolerance so faults don't become failures. You can only tolerate *certain* fault types — name them.

Fault classes: Hardware (random, uncorrelated) · Software (systematic, correlated across nodes, dormant until unusual trigger) · Human (config errors = leading cause of outages; hardware only 10–25%). At 10k disks (MTTF 10–50 years), expect ~one disk death per day. Cloud VMs vanish without warning.

3. Framework: Load Parameters & Scalability Method

You cannot say "X is scalable." You must ask: **"If the system grows in a particular way, what are our options for coping?"**

When	Any capacity/scaling question — especially interviews. State load parameters first.
How	**(1)** Describe current load with a few numbers — **load parameters** (requests/sec, read/write ratio, active users in a chat room, cache hit rate, follower distribution). Pick the ones your bottleneck depends on; extreme cases often dominate, not the average. **(2)** Describe performance: throughput for batch; **response time distribution** (percentiles, not means) for online. **(3)** Ask: with resources fixed, how does performance degrade as a load parameter grows? How much resource to hold performance constant? **(4)** Choose coping: scale up vs scale out, elastic vs manual.
Why it works	Architectures assume which operations are common vs rare. Wrong assumptions = wasted or counter-productive scaling effort. Expect to rethink architecture roughly every order-of-magnitude load increase.
Failure mode	Attaching "scalable" as a one-dimensional label without stating which load parameter grows.

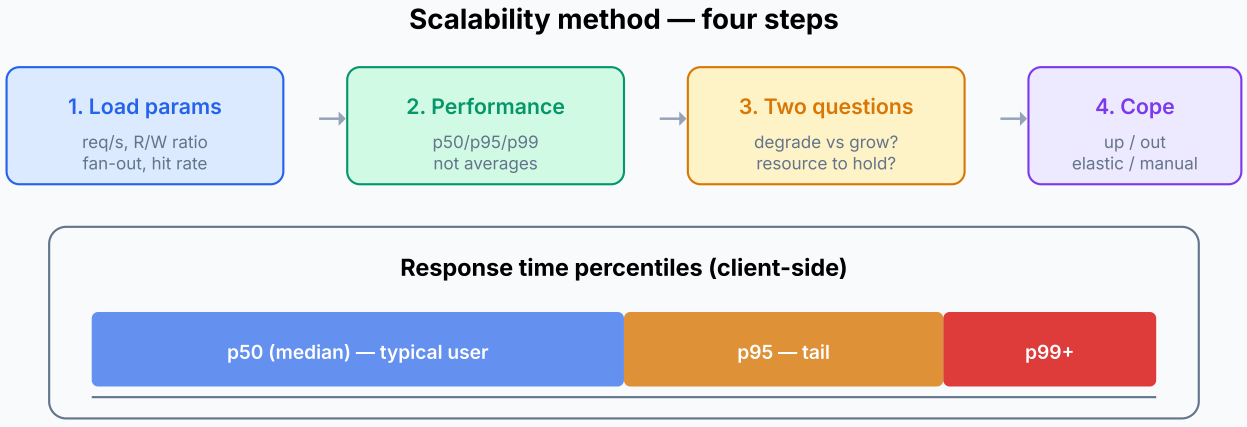

Figure 3.1 — Name load parameters, measure percentiles, then choose coping strategy.

Scale up vs scale out: Vertical (bigger machine) vs horizontal (**shared-nothing** across machines). Good architectures pragmatically mix both. Keep stateful data on a single node until cost or HA forces distribution; stateless services distribute easily.

No magic scaling sauce: 100k req/s of 1 kB differs completely from 3 req/min of 2 GB despite equal throughput. 100 ms slower cost Amazon 1% of sales; a 1 s slowdown cuts a satisfaction metric 16%.

4. Framework: Maintainability Triad

Operability / Simplicity / Evolvability — three design principles to avoid creating legacy software. Most software cost is maintenance, not initial development.

Principle	What it means	How to achieve it
Operability	Make routine ops easy for the team running the system	Visibility/monitoring, automation support, no single-machine dependency, good defaults with overrides, predictable behavior
Simplicity	Remove **accidental complexity** — complexity not inherent in the problem, only in the implementation (Moseley & Marks)	Good **abstractions** — e.g., SQL hides on-disk structures, concurrency, crash recovery
Evolvability	Make change easy — agility at the data-system level	Tightly linked to simplicity; easy-to-modify systems require simple foundations

Guard against humans, the leading outage cause: minimize error opportunities via good abstractions, decouple sandboxes (real data, no real users) from production, test at all levels, make rollback fast and rollouts gradual, instrument telemetry.

5. Key Concepts

Term	Definition
Fault vs failure	Fault = one component deviating from spec; failure = whole system stops serving. Design fault-tolerance so faults don't become failures.
Fault-tolerant / resilient	Anticipates and copes with certain types of faults — never all possible faults.
Load parameters	The few numbers that describe load on your system; the right ones depend on your architecture.
Fan-out	Number of requests to other services needed to serve one incoming request (borrowed from electronics).
Response time vs latency	Response time = what the client sees (service time + network + queueing); latency = time a request waits to be handled.
Percentiles (p50/p95/p99/p999)	Response-time thresholds that a given fraction of requests beat; median = typical, high percentiles = tail experience.
Head-of-line blocking	A few slow requests hold up subsequent ones due to limited parallelism, inflating client-observed response times.
Tail latency amplification	With multiple parallel backend calls per user request, the request waits for the slowest call — rare slow backend calls make many end-user requests slow.
Scaling up vs out	Vertical (bigger machine) vs horizontal (**shared-nothing** across machines); good architectures mix both.
Accidental complexity	Complexity from implementation, not the problem itself — the target of simplification.

6. Mental Models

1. Reliability = enumerate fault classes

"Continuing to work correctly even when things go wrong." Hardware (random), software (systematic, correlated), human (config errors #1 cause).

2. Chaos engineering

Deliberately increase fault rate (Netflix Chaos Monkey). Many critical bugs are poor error handling — only exercised fault-tolerance machinery is trusted machinery.

3. Percentiles, never averages

Mean doesn't tell you how many users experienced a delay. Never average percentiles across machines/time — aggregate by adding histograms (forward decay, t-digest, HdrHistogram).

4. Write-time vs read-time dial

When reads outnumber writes by orders of magnitude, precompute at write time (fan-out) even though it multiplies write work.

7. Anti-Patterns

Anti-pattern	Why it fails
Attaching "scalable" as a one-dimensional label	Meaningless without stating which load parameter grows and what coping options exist
Reporting mean response time	Hides the tail that your most valuable users hit (Amazon: slowest requests often from customers with most data/purchases)
Averaging percentiles	Mathematically meaningless; add histograms instead
Load-testing with sequential client	Client waits for each response before sending next → artificially short queues, skewed measurements. Load generator must send independently of response time.
Chasing extreme tails blindly	Amazon targets p999 but judged optimizing p9999 too expensive — tail costs rise while benefits diminish
Interfaces so restrictive people work around them	Minimizing human error via lockdown backfires; balance "easy to do the right thing" against escape hatches
Premature scaling in unproven product	Iterating on features usually beats scaling for hypothetical load; architecture built on wrong load-parameter assumptions is wasted effort

8. Worked Example: Twitter Fan-Out (Nov 2012)

Load: post tweet averages 4.6k req/s (12k peak); home-timeline reads 300k req/s. 12k writes/sec alone is easy — the challenge is **fan-out** (each user follows and is followed by many).

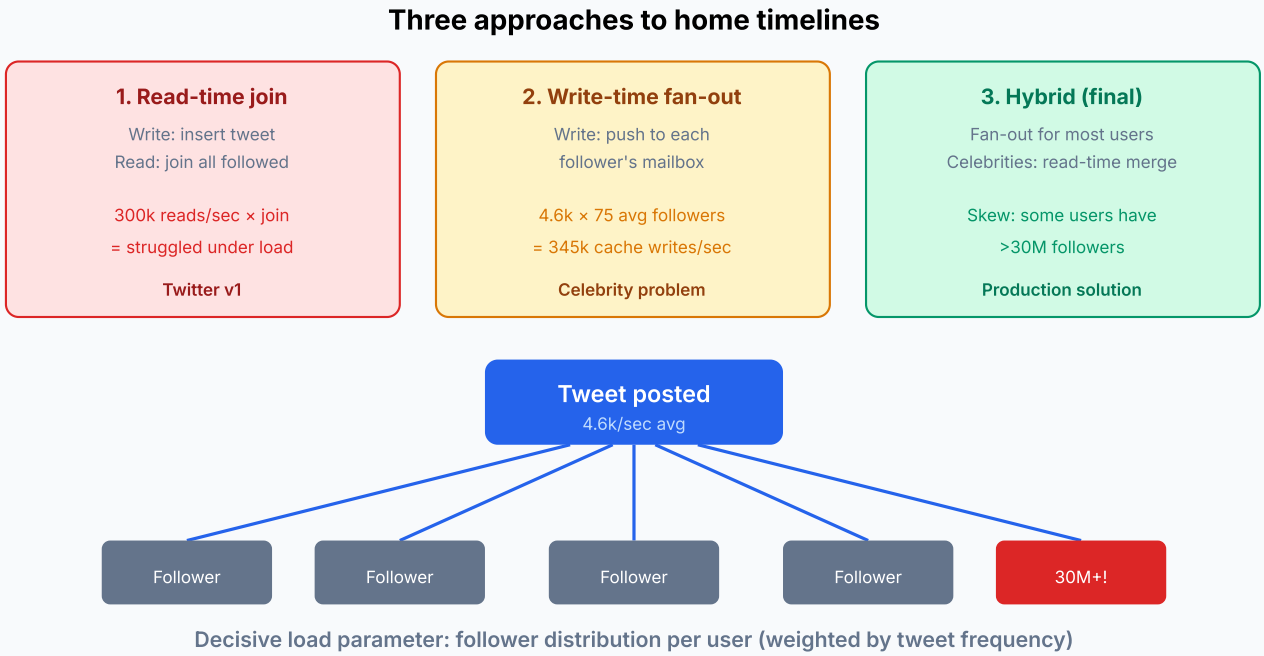

Figure 8.1 — Write-time fan-out works when reads » writes, but celebrity skew breaks naive fan-out.

Approach 1: Read-time join

Posting inserts into a global tweets collection; reading a timeline joins tweets from everyone the user follows. Twitter's first version — struggled under home-timeline read load.

Approach 2: Write-time fan-out

Keep a per-user timeline cache (mailbox); posting inserts the tweet into every follower's cache, making reads cheap. Works because tweet rate is ~two orders of magnitude below read rate. Cost: average 75 followers turns 4.6k tweets/sec into 345k cache writes/sec. Distribution wildly skewed — some users have >30M followers, so one tweet can mean >30M writes against a ~5-second delivery target.

Hybrid (final)

Fan out for most users, but exempt celebrities; their tweets are fetched at read time and merged in — consistently good performance.

Key lesson: The follower distribution per user (weighted by tweet frequency) is the decisive load parameter. The write-time vs read-time trade-off dial: when reads outnumber writes by orders of magnitude, precompute at write time even though it multiplies write work.

9. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Load parameters + write/read rate skew → write-time fan-out decision	Read rate outnumbers write rate by orders of magnitude; naive read-time join can't keep up	News feed system (Twitter fan-out example)
p99 / tail-latency focus, GC pause named as a threat	A single stop-the-world pause blows a millisecond-level SLA on the hot path	Stock exchange (round-trip p99 target, GC pause called out explicitly)
Availability nines + cost-of-downtime math	Justify infra spend (CDN, redundancy) against concrete dollar cost of an outage	Youtube-style video platform ($150k/day CDN cost framing)
Master→slave replication + resharding as scale-out response	Single-node capacity exhausted; must add machines without a rewrite	Generic scaling walkthrough (master-slave, sharding by key)
Back-of-envelope load/availability estimation	Size a system before building — is this 100 req/s or 100k req/s?	Any capacity-estimation pass before choosing a Ch 6/7/9 mechanism

10. Connects To (Cross-Chapter)

Chapter / Resource	Connection	Interview soundbite
Ch 2	Data models and query languages are the first layer of abstraction chosen when composing a data system.	"SQL is an abstraction that hides on-disk structures and crash recovery."
Ch 5–6 (Replication/Partitioning)	The scale-out / shared-nothing path introduced here; rebalancing partitions revisits elastic vs manual scaling.	"Scale out stateless first; stateful adds real complexity."
Part III (Derived data)	The Figure 1-1 composite system (DB + cache + index + queue kept in sync by application code) is the problem those chapters systematize.	"One system of record + CDC, not dual-write."
The Tail at Scale (Dean & Barroso)	Canonical treatment of tail-latency amplification across parallel backend calls.	"Parallel backends → user waits for slowest; p99 matters."
SRE practice	SLOs/SLAs, telemetry, chaos engineering, and gradual rollouts are the operational embodiment of reliability and operability principles.	"SLOs in percentiles; chaos tests fault-tolerance machinery."

11. Key Takeaways

1. **Design fault-tolerance** so component faults don't become system failures; you can only tolerate certain fault types, so name them (hardware, software, human).
2. **Prefer software fault-tolerance** over pure hardware redundancy at scale — with 10k disks expect roughly one disk death per day; cloud VMs vanish without warning. Machine-loss tolerance enables rolling patches with zero downtime.
3. **Guard against humans**, the leading outage cause: minimize error opportunities via good abstractions, decouple sandboxes from production, test at all levels, make rollback fast and rollouts gradual, instrument telemetry.
4. **Start every scalability discussion** by naming load parameters; know whether averages or extreme cases drive your bottleneck.
5. **Measure response time as a distribution** on the client side; set SLOs/SLAs in percentiles (e.g., median < 200 ms and p99 < 1 s, up 99.9% of the time).
6. **Keep stateful data on a single node** (scale up) until cost or HA forces distribution; stateless services distribute easily. There is no magic scaling sauce.
7. **Optimize for maintenance**, the majority of software cost: build in operability, cut accidental complexity with abstractions, and design for evolvability.

12. Junior SWE Checkpoints

Q1: What's the difference between a fault and a failure?

A **fault** is one component deviating from spec (disk crash, bug, config typo). A **failure** is the whole system stopping to provide service. Design goal: fault-tolerance so faults don't become failures.

Q2: Why can't you say "PostgreSQL is scalable"?

Scalability is not a yes/no label. You must state **which load parameter grows** (req/s? data volume? fan-out?) and **what coping options exist** (scale up, replicate, shard, cache).

Q3: Why use p99 instead of mean response time?

Mean hides the tail. Your most valuable users often hit the slowest requests (Amazon finding). p99 tells you what fraction of users experience delays. Never average percentiles across machines — add histograms.

Q4: Twitter fan-out — why did write-time fan-out fail for celebrities?

Skewed follower distribution: average 75 followers is fine (4.6k × 75 = 345k writes/sec), but users with >30M followers mean one tweet triggers >30M writes against a ~5s delivery target. Hybrid: fan-out for normal users, read-time merge for celebrities.

Q5: What is accidental complexity?

Complexity from the **implementation**, not the problem itself (Moseley & Marks). Target of simplification. Good abstractions (SQL) hide accidental complexity from users.

Q6: What is tail latency amplification?

When a user request fans out to multiple parallel backend calls, the end-user waits for the **slowest** call. Even rare slow backend calls make many user requests slow. See "The Tail at Scale" (Dean & Barroso).

Glossary

Term	Definition
Load parameter	A number describing load — req/s, R/W ratio, fan-out, cache hit rate
Fan-out	Requests to other services needed to serve one incoming request
Percentile (p50/p95/p99)	Response-time threshold that a given fraction of requests beat
Scale up / scale out	Vertical (bigger machine) vs horizontal (shared-nothing cluster)
Accidental complexity	Implementation complexity not inherent in the problem
Fault-tolerance	Coping with certain fault types so they don't cause system failure
Operability	Making routine ops easy — monitoring, automation, predictable behavior
Evolvability	Making change easy over the system's lifetime
Write-time fan-out	Precompute at write time (push to followers) when reads ≫ writes
Tail latency amplification	Parallel backends → user waits for slowest call